



Furni Store

Backend API Documentation

GET /api/products · POST /api/login · POST /api/register

Furniture E-commerce Platform · Controller · Route · Service Pattern

Node.js · Express · bcryptjs · jsonwebtoken

PROJECT	Furni Furniture Online Store
MODULE	Product Catalog · Authentication (Login & Register)
STACK	Node.js · Express 4 · bcryptjs 2 · jsonwebtoken 9 · CORS
DATA	data/json/products.json · users.json · auth_user.json
AUTHOR	Veerachit Mongkol
DATE	May 2026

GenAI Prompts

The three prompts below were used to generate the products filter, the registration flow, and the login route. They are recorded here verbatim so the generation steps are reproducible and auditable.

PROMPT 1 — PRODUCTS FILTER (GET /API/PRODUCTS)

You are a backend assistant. Build an Express route for `GET /api/products`. The request envelope includes a query parameter: `category`. Implement gatekeeper logic: allow only when `category` equals `"hat"` (case-insensitive). On success, read local `products.json`, filter products by category (match in title), and respond with JSON `{ status: "success", data: Product[] }`. On failure (gatekeeper or server error), respond with JSON `{ status: "fail", message }`. Add brief comments in code explaining envelope, gatekeeper, and package.

■ *Implementation note: The gatekeeper rejects any non-"hat" category with a 400. Filtering uses a case-insensitive substring match against `title`, so a future `category=chair` would also work if the gatekeeper were relaxed.*

PROMPT 2 — USER REGISTRATION (POST /API/REGISTER)

You are a full-stack assistant. Implement user registration. **Backend:** Express `POST /api/register`. Expect JSON body `{ name, email, password }`. Check if email already exists in `auth_user.json`. If exists, return 409 with `{ status: "fail", message }`. If new, hash password with `bcrypt` and store user in `auth_user.json`. Return 201 `{ status: "success", message }`. **Frontend:** `register.js` should validate password rules before sending: at least 8 characters, at least 1 uppercase, at least 1 special character (`!@#$$%^&*`). Show clear error messages and redirect to login on success. Add brief comments explaining the flow in both route and `register.js`.

■ *Implementation note: `bcryptjs` is used with the default cost factor (10). Two gatekeepers run in series — the frontend gatekeeper blocks weak passwords before any network call, and the backend gatekeeper re-validates and checks for duplicates because the client cannot be trusted.*

PROMPT 3 — USER LOGIN (POST /API/LOGIN)

Act as a Security Engineer. Write a Node.js/Express `POST` route for `/api/login`. Receive `{ email, password }` in the request body. If either field is missing, return 400. Look up the user in `data/json/users.json` by `username` (= email). If the user is not found, return 401 Unauthorized. If found, hash the submitted password with MD5 (assignment requirement) and compare with the stored hash. On mismatch, return 401. On match, sign a JWT containing `{ userId }` using `JWT_SECRET` from `process.env` (fallback `"dev-secret"`) with `expiresIn: "1h"`, and return `{ status: "success", token }`. Wrap the handler in `try/catch` and return 500 with `{ status: "fail", message }` on unexpected errors.

■ *Implementation note: MD5 is used here only because the assignment specifies it for the seed `users.json` dataset. In production, MD5 is unsuitable for password storage — `bcrypt` (used in registration) should be the standard. JWTs are signed server-side; the secret must come from an environment variable, never from source code.*

API Contract Table

All three endpoints exposed by the Furni backend. Each row defines the method, path, request shape, and every possible response status with its payload schema.

2.1 Endpoint Overview

Method	Route	Auth Required	Purpose
GET	/api/products	No	List products filtered by <code>?category=hat</code>
POST	/api/login	No	Authenticate user (MD5 + JWT issuance)
POST	/api/register	No	Create new user account (bcrypt hash)

2.2 GET /api/products — Full Contract

REQUEST ENVELOPE (QUERY STRING)

Field	Type	Required	Validation Rules
category	string	Yes	Must equal <code>"hat"</code> (case-insensitive)

RESPONSE CODES

Status	Condition	Response Body
200 OK	Gatekeeper passes; products read successfully	<code>{ "status": "success", "data": Product[] }</code>
400 Bad Request	Gatekeeper rejects (category missing or <code>≠ "hat"</code>)	<code>{ "status": "fail", "message": "Gatekeeper rejected request" }</code>
500 Internal Server Error	products.json read or parse failure	<code>{ "status": "fail", "message": "Failed to load products" }</code>

PRODUCT SCHEMA (FROM PRODUCTS.JSON)

Field	Type	Example
id	string	<code>p-001</code>
href	string	<code>cart.html</code>
image	string	<code>images/product-1.png</code>
title	string	<code>Nordic Chair</code>
price	string	<code>\$50.00</code>
icon	string	<code>images/cross.svg</code>

2.3 POST /api/register — Full Contract

REQUEST BODY (CONTENT-TYPE: APPLICATION/JSON)

Field	Type	Required	Validation Rules
name	string	Yes	Non-empty after trim
email	string	Yes	Used as username; must not exist in <code>auth_user.json</code>
password	string	Yes	Min 8 chars · ≥ 1 uppercase · ≥ 1 special (<code>!@#\$\$%^&*</code>)

RESPONSE CODES

Status	Condition	Response Body
201 Created	Backend gatekeeper passes, email is new, bcrypt hash written to <code>auth_user.json</code>	<code>{ "status": "success", "message": "Registration successful" }</code>
400 Bad Request	Missing field or password fails strength rules	<code>{ "status": "fail", "message": "Password must be 8+ chars with 1 uppercase and 1 special (!@#\$\$%^&*)" }</code>
409 Conflict	Email already exists in <code>auth_user.json</code>	<code>{ "status": "fail", "message": "Email already registered" }</code>
500 Internal Server Error	JSON read/write failure or bcrypt error	<code>{ "status": "fail", "message": "Registration failed" }</code>

AUTH_USER.JSON SCHEMA

Field	Type	Example
id	string	<code>au-1714900000000</code>
name	string	<code>Nina Wood</code>
username	string	<code>nina.wood@example.com</code>
password	string (bcrypt)	<code>\$2a\$10\$...</code>
createdAt	string (YYYY-MM-DD)	<code>2026-05-05</code>

2.4 POST /api/login — Full Contract

REQUEST BODY (CONTENT-TYPE: APPLICATION/JSON)

Field	Type	Required	Validation Rules
email	string	Yes	Looked up against <code>users.json</code> by <code>username</code>
password	string	Yes	MD5-hashed server-side, compared with stored hash

RESPONSE CODES

Status	Condition	Response Body
200 OK	Email found <i>and</i> MD5 hash matches	<code>{ "status": "success", "token": "<JWT>" }</code>
400 Bad Request	Missing email or password	<code>{ "status": "fail", "message": "Email and password are required" }</code>
401 Unauthorized	Email not found <i>or</i> hash mismatch (intentionally same message)	<code>{ "status": "fail", "message": "Unauthorized" }</code>
500 Internal Server Error	<code>users.json</code> read failure or <code>jwt.sign</code> failure	<code>{ "status": "fail", "message": "Login failed" }</code>

Architecture Diagrams

Controller–Route–Service file layout, followed by the contract + sequence diagrams for each endpoint.

3.1 Controller–Route–Service File Structure

Layer	File	Responsibility
Entry	src/server.js	Bind to PORT 3000; import app.js
Entry	src/app.js	Express app: JSON body parser, CORS, mount /api/* routes
Route	src/routes/products.js	Declare GET /products → controller
Route	src/routes/auth.js	Declare POST /login, POST /register
Controller	src/controllers/productsController.js	Envelope read · gatekeeper (category=="hat") · package response
Controller	src/controllers/authController.js	Login: MD5 compare + jwt.sign · Register: strength check + bcrypt.hash
Service	src/services/productsService.js	Read products.json · filter by category in title
Service	src/services/authService.js	Lookup/insert in users.json & auth_user.json
Data	data/json/products.json	Mock product catalog
Data	data/json/users.json	Seed login users (MD5 hashes)
Data	data/json/auth_user.json	Live-write registered users (bcrypt hashes)

3.2 GET /api/products — Contract & Sequence Diagram

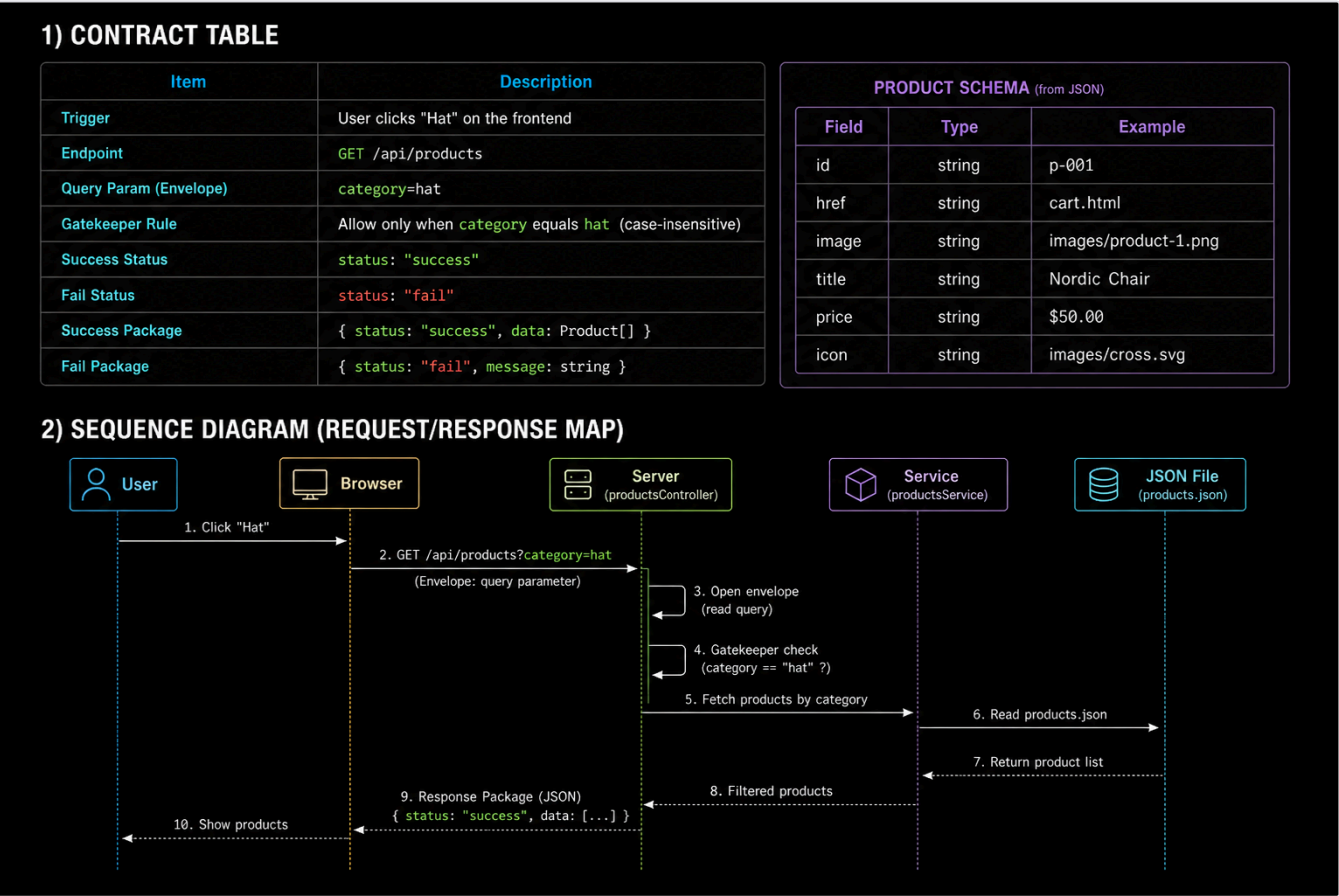


Fig 1. Contract table + sequence map for GET /api/products?category=hat . Browser → Server (envelope & gatekeeper) → Service → JSON file → packaged response.

3.3 POST /api/register — Contract & Sequence Diagram

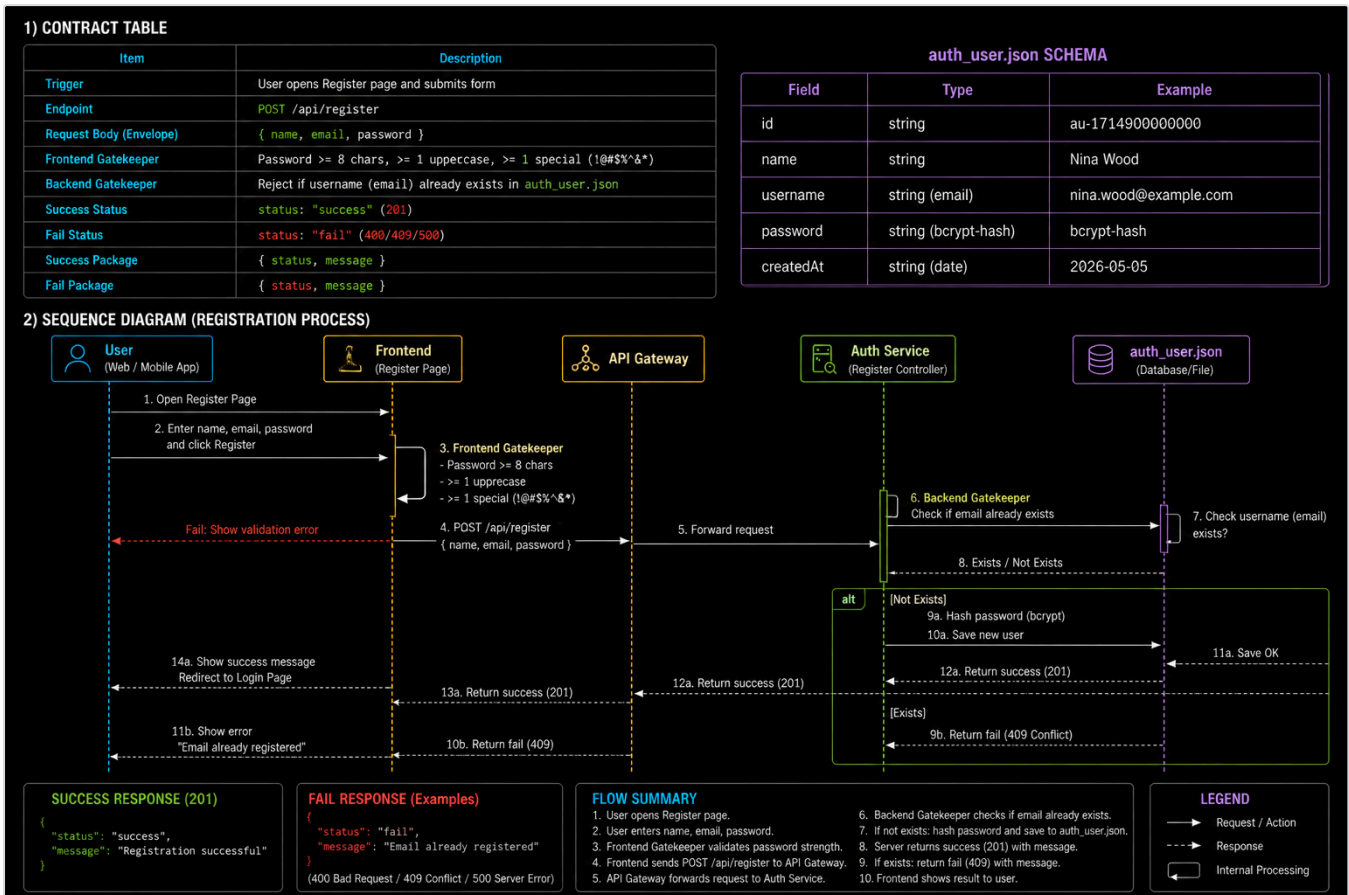


Fig 2. Contract table + UML sequence for POST /api/register . Two gatekeepers (frontend strength check, backend

3.4 POST /api/register — Activity & Sequence Diagram

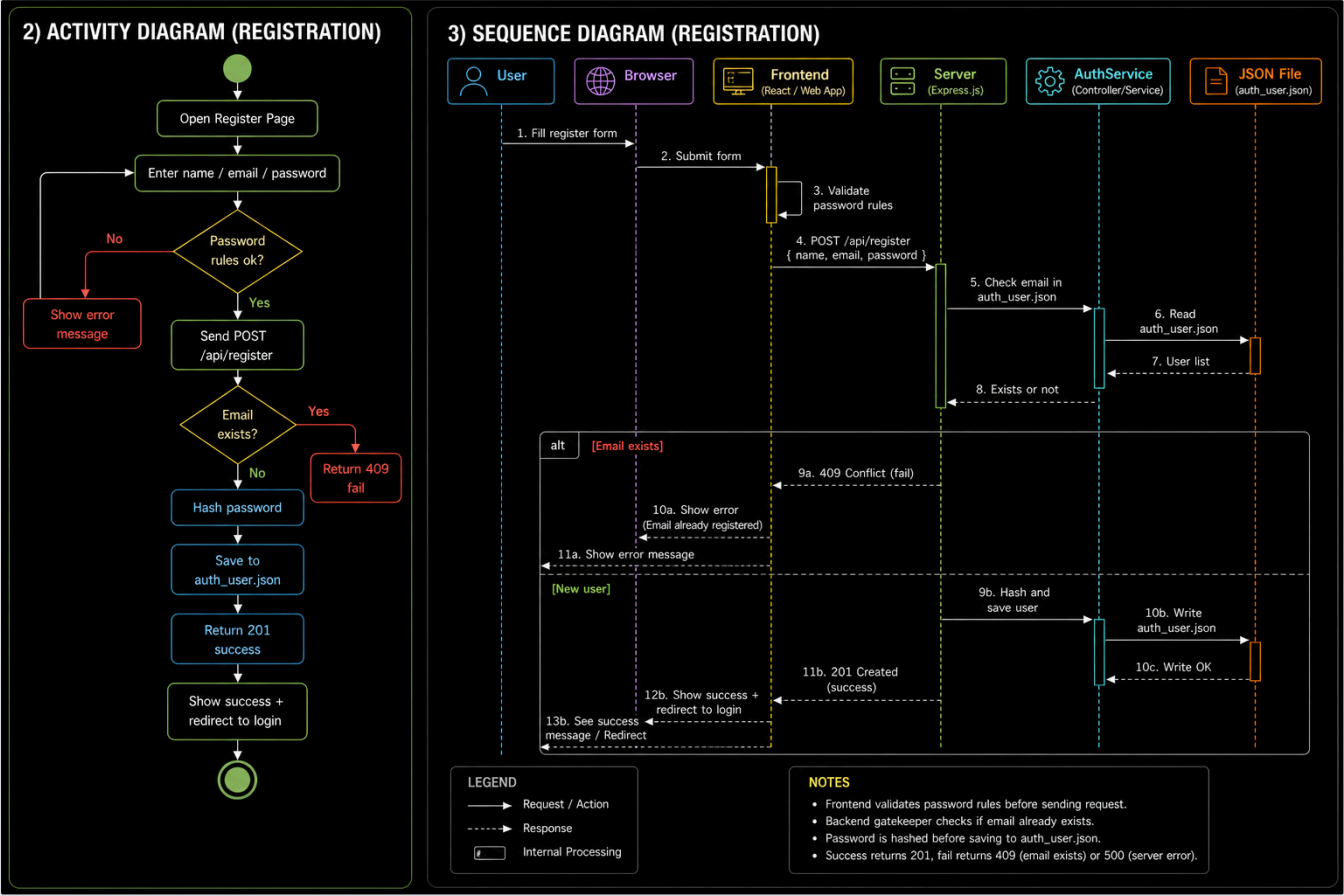


Fig 3. Activity diagram (left) and full UML sequence (right) for the registration flow. Alt frames cover the 409 duplicate-email branch and the 201 success path.

duplicate check) precede bcrypt hashing and the JSON write.

3.5 POST /api/login — UML Sequence Diagram

Sequence diagram for the login flow. Yellow numbered tags mark each message step. Alt frames cover the 400 missing-field branch, the 401 user-not-found / hash-mismatch branches, and the 200 success path with JWT issuance.

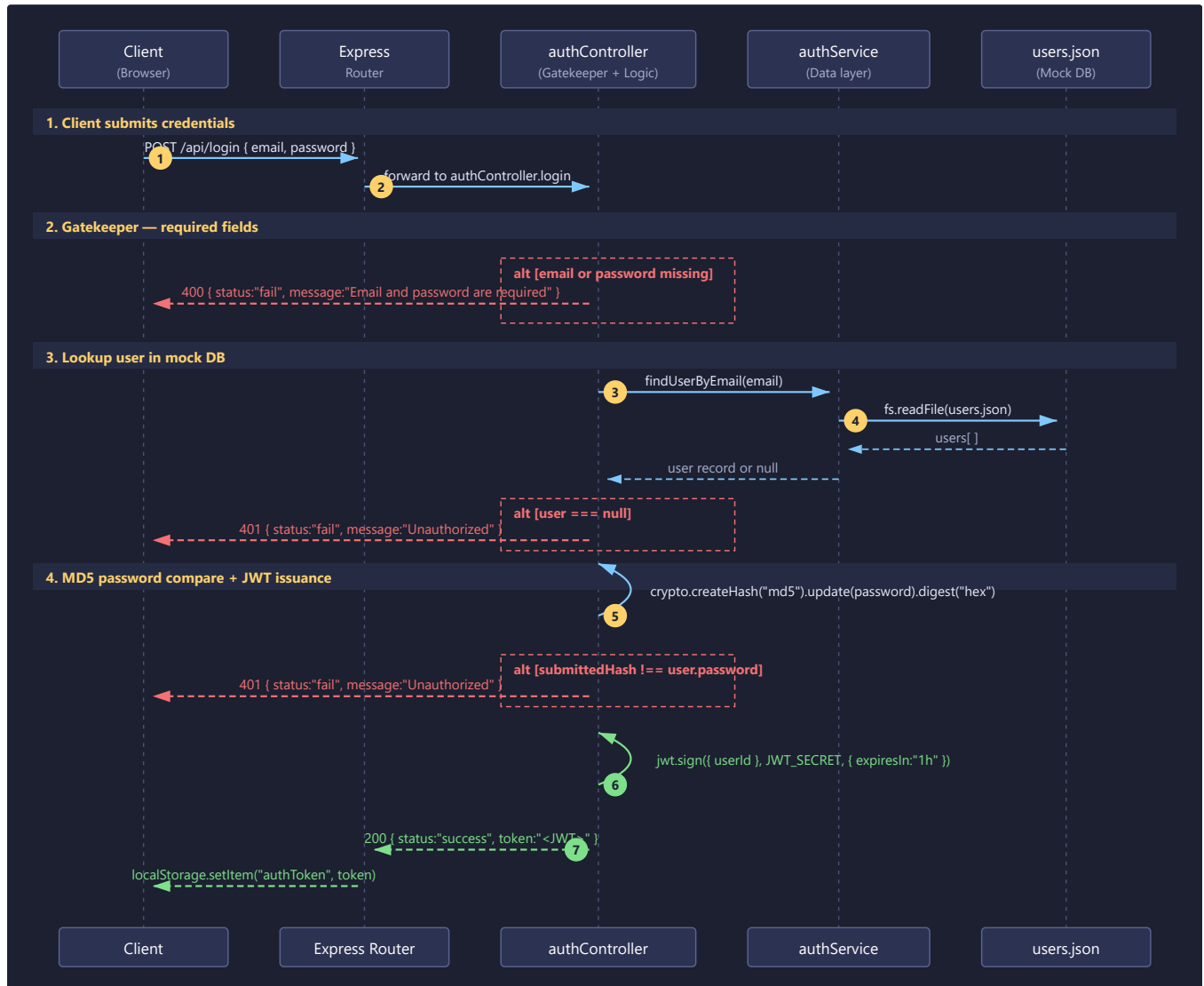


Fig 4. POST /api/login sequence — Client → Router → authController (gatekeeper + MD5 compare) → authService → users.json. Red dashed returns are failure paths (400 / 401); green dashed returns are the success path (200) carrying the signed JWT.

3.6 Security & Design Decisions

Decision	Reason
Two gatekeepers for register (frontend + backend)	The frontend gatekeeper (<code>register.js</code>) gives instant feedback and avoids unnecessary requests, but the client cannot be trusted — the backend re-validates the same rules and additionally checks for duplicate email.
bcrypt for register, MD5 for login (legacy seed data)	New accounts in <code>auth_user.json</code> use bcrypt with cost factor 10 (slow + per-hash salt → resists brute-force). The seed dataset <code>users.json</code> uses MD5 only because the assignment specifies it; this is documented as a known weakness, not a recommendation.
Same 401 message for "user not found" and "wrong password"	Returning distinct messages would let an attacker enumerate which emails are registered. A single generic "Unauthorized" response leaks nothing.
JWT payload contains only { <code>userId</code> }	JWTs are Base64-decodable by anyone holding the token. Sensitive fields (password hash, email, address) must never appear in the payload.
JWT_SECRET from <code>process.env</code>	Hardcoding secrets in source exposes them in git history. The "dev-secret" fallback is for local dev only and must be overridden in any deployed environment.
CORS allowlist (<code>127.0.0.1:5500</code> / <code>localhost:5500</code>)	The Live Server origin is explicitly whitelisted instead of using <code>"*"</code> . This blocks cross-origin requests from arbitrary sites attempting to call the API on a logged-in user's behalf.
Envelope / Gatekeeper / Package vocabulary in code comments	Each controller and service carries one-line comments labelling these three phases. This matches the conceptual model taught in the WebTech course and makes the intent of each block visible at a glance during code review.

— End of report —